

Aluno: Thiago Henrique de Campos Silva
R.A: 24019349-2
Turma:ESOFT6S-N-C

01 – Lexemas-token

IDENTIFICADORES – valorFinal, somaAnterior, valor

```
4 public class Main {
5     static final String ALUNO = "THIAGO HENRIQUE DE CAMPOS SILVA";
6     static final String RA = "24019349-2";
7
8     static final String ENTRADA = "valorFinal = somaAnterior - valor / 500;";
9
10    private static final Map<Character, String> SIMBOLOS = new LinkedHashMap<>();
11
12    static {
13        SIMBOLOS.put('-', "ATRIBUICAO");
14        SIMBOLOS.put('+', "SOMA");
15        SIMBOLOS.put('-', "SUBTRACAO");
16        SIMBOLOS.put('*', "MULTIPLICACAO");
17        SIMBOLOS.put('/', "DIVISAO");
18        SIMBOLOS.put('.', "PONTO_E_VIRGULA");
19        SIMBOLOS.put('(', "ABRE_PARENTESES");
20        SIMBOLOS.put(')', "FECHA_PARENTESES");
21    }
22
23    public static void main(String[] args) {
24        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
25        System.out.println("ENTRADA: " + ENTRADA);
26        System.out.println();
27        System.out.printf("%-22s %s\n", "LEXEMA", "TOKEN");
28        System.out.println("-----");
29
30        analisar(ENTRADA);
31    }
```

```
ALUNO: THIAGO HENRIQUE DE CAMPOS SILVA | RA: 24019349-2
ENTRADA: valorFinal = somaAnterior - valor / 500;
```

LEXEMA	TOKEN
valorFinal	IDENTIFICADOR
=	ATRIBUICAO
somaAnterior	IDENTIFICADOR
-	SUBTRACAO
valor	IDENTIFICADOR
/	DIVISAO
500	INTEIRO
;	PONTO_E_VIRGULA

02 - Classes reservadas

IDENTIFICADOR – inteiro, TH, valor

PALAVRA_RESERVADA – int

ERRO_LEXICO - #

```
1 import java.util.Arrays;
2 import java.util.HashSet;
3 import java.util.Set;
4
5 public class Main {
6     static final String ALUNO = "THIAGO HENRIQUE DE CAMPOS SILVA";
7     static final String RA = "24019349-2";
8
9     static final String IDENTIFICADOR_PERSONALIZADO = "TH";
10
11     static final Set<String> PALAVRAS_RESERVADAS = new HashSet<>()
12     Arrays.asList("int", "if", "else", "while", "return");
13 }
14
15 public static void main(String[] args) {
16     System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
17     executarTeste("TESTE A - entrada compacta",
18         "int total=valor+10;");
19     executarTeste("TESTE B - espaços e comentário",
20         "int total = valor + 10; // este comentario sera ignorado");
21     executarTeste("TESTE C - reservadas e identificadores",
22         "int inteiro = 0; while (inteiro < intValue) inteiro = inteiro + 1;");
23     executarTeste("TESTE C2 - identificador personalizado",
24         "int " + IDENTIFICADOR_PERSONALIZADO + " = 10;");
25     executarTeste("TESTE D - caractere inválido",
26         "int valor# = 1;");
27 }
28
```

inteiro	LETRA	<u>IDENTIFICADOR</u>
+	SIMBOLO	SOMA
1	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

=== TESTE C2 - identificador personalizado ===

ENTRADA: int TH = 10;	LEXEMA	CLASSE	TOKEN
int	LETRA	<u>PALAVRA_RESERVADA</u>	
TH	LETRA	<u>IDENTIFICADOR</u>	
=	SIMBOLO	ATRIBUICAO	
10	DIGITO	INTEIRO	
;	SIMBOLO	PONTO_E_VIRGULA	

=== TESTE D - caractere inválido ===

ENTRADA: int valor# = 1;	LEXEMA	CLASSE	TOKEN
int	LETRA	<u>PALAVRA_RESERVADA</u>	
valor	LETRA	<u>IDENTIFICADOR</u>	
#	DESCONHECIDO	<u>ERRO_LEXICO</u>	
=	SIMBOLO	ATRIBUICAO	
1	DIGITO	INTEIRO	
;	SIMBOLO	PONTO_E_VIRGULA	

03 – Diagnósticos

Compilando com erro

```
1 public class Main {
2     static final String ALUNO = "THIAGO HENRIQUE DE CAMPOS SILVA";
3     static final String RA = "24019349-2";
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula
9
10        if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
12            System.out.println("Resultado: " + total);
13        }
14    }
15 }
16
17
18
19
20
```

```
Main.java:8: error: ';' expected
    int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e
    virgula
                    ^
Main.java:10: error: ')' expected
    if (total > 20 { // CORRIGIR 2: feche corretamente o
    parenteses
                ^
Main.java:11: error: unclosed string literal
        System.out.println("Programa corrigido!"); // CORRIGIR 3:
        feche o texto entre aspas
                        ^
3 errors
Debug this error
```

Aluno: Thiago Henrique de Campos Silva
R.A: 24019349-2
Turma:ESOFT6S-N-C

Código com erro

```
1 public class Main {
2     static final String ALUNO = "THIAGO HENRIQUE DE CAMPOS SILVA";
3     static final String RA = "24019349-2";
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20;
9
10        if (total > 20) {
11            System.out.println("Programa corrigido!");
12            System.out.println("Resultado: " + total);
13        }
14    }
15 }
16
17
18
19
20
```

ALUNO: THIAGO HENRIQUE DE CAMPOS SILVA | RA: 24019349-2
Programa corrigido!
Resultado: 30

04 – Parser – descendente

```
1 public class Main {
2     static final String ALUNO = "THIAGO HENRIQUE DE CAMPOS SILVA";
3     static final String RA = "24019349-2";
4
5     static final String ENTRADA = "(5 + 3) * 4";
6     static final boolean DEMONSTRAR_RECURSAO_ESQUERDA = false;
7
8     public static void main(String[] args) {
9         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
10        System.out.println("ENTRADA: " + ENTRADA);
11        System.out.println();
12
13        if (DEMONSTRAR_RECURSAO_ESQUERDA) {
14            demonstrarRecursoEsquerda(0);
15            System.out.println();
16        }
17
18        try {
19            Parser parser = new Parser(ENTRADA);
20            int resultado = parser.analisar();
21            System.out.println();
22            System.out.println("ENTRADA ACEITA");
23            System.out.println("RESULTADO DA EXPRESSAO: " + resultado);
24        } catch (ErroSintatico erro) {
25            System.out.println();
26            System.out.println("ERRO SINTATICO: " + erro.getMessage());
27        }
28    }
29 }
```

SAINDO de <factor> | lookahead = +
SAINDO de <term> | lookahead = +
Encontrado operador +
Novo lookahead: 3
ENTRANDO em <term> | lookahead = 3
ENTRANDO em <factor> | lookahead = 3
Numero reconhecido: 3
Novo lookahead:)
SAINDO de <factor> | lookahead =)
SAINDO de <term> | lookahead =)
SAINDO de <expr> | lookahead =)
Novo lookahead: *
Fechando parenteses
SAINDO de <factor> | lookahead = *
Encontrado operador *
Novo lookahead: 4
ENTRANDO em <factor> | lookahead = 4
Numero reconhecido: 4
Novo lookahead: \$FIM\$
SAINDO de <factor> | lookahead = \$FIM\$
SAINDO de <term> | lookahead = \$FIM\$
SAINDO de <expr> | lookahead = \$FIM\$

ENTRADA ACEITA
RESULTADO DA EXPRESSAO: 32

05 - Parser-ascendente

Aluno: Thiago Henrique de Campos Silva
R.A: 24019349-2
Turma:ESOFT6S-N-C

```
1 import java.util.ArrayList;
2 import java.util.HashMap;
3 import java.util.List;
4 import java.util.Map;
5
6 public class Main {
7     static final String ALUNO = "THIAGO HENRIQUE DE CAMPOS SILVA";
8     static final String RA = "24019349-2";
9
10    static final String ENTRADA_VALIDA = "Id*Id*Id";
11    static final String ENTRADA_INVALIDA = "Id*Id";
12
13    // Na primeira execução, use ENTRADA_VALIDA. Depois troque por ENTRADA_INVALIDA.
14    static final String ENTRADA = ENTRADA_VALIDA;
15
16    static final Map<Integer, Map<String, String>> ACTION = new HashMap<>();
17    static final Map<Integer, Map<String, Integer>> GOTO = new HashMap<>();
18    static final Map<Integer, Regra> REGRAS = new HashMap<>();
19
20    static {
21        // Gramática:
22        // 1) E -> E + T      2) E -> T
23        // 3) T -> T * F      4) T -> F
24        // 5) F -> ( E )      6) F -> Id
25        regra(1, "E", 3, "E -> E + T");
26        regra(2, "E", 1, "E -> T");
27        regra(3, "T", 3, "T -> T * F");
28        regra(4, "T", 1, "T -> F");
29        regra(5, "F", 3, "F -> ( E )");
30        regra(6, "F", 1, "F -> Id");
31
32        action(0, "Id", "s5"); action(0, "(", "s4");
33        action(1, "+", "s6"); action(1, "$", "acc");
34        action(2, "+", "r2"); action(2, "**", "s7");
35        action(3, ")", "r2"); action(3, "$", "r3");
36        action(3, "+", "r4"); action(3, "**", "r4");
37        action(3, ")", "r4"); action(3, "$", "r4");
38        action(4, "Id", "s5"); action(4, "(", "s4");
39        action(5, "+", "r4"); action(5, "**", "r4");
40        action(5, ")", "r4"); action(5, "$", "r4");
41        action(6, "Id", "s5"); action(6, "(", "s4");
42        action(7, "Id", "s5"); action(7, "(", "s4");
43        action(8, "+", "s6"); action(8, ")", "s11");
44        action(9, "+", "r1"); action(9, "**", "s7");
45        action(9, ")", "r1"); action(9, "$", "r1");
46        action(10, "+", "r3"); action(10, "**", "r3");
```

ALUNO: THIAGO HENRIQUE DE CAMPOS SILVA | RA: 24019349-2

ENTRADA: Id*Id*Id

PILHA	ENTRADA	ACAO
\$ 0	Id*Id*Id\$	SHIFT Id -> estado 5
\$ 0 Id 5	+Id*Id\$	REDUCE F -> Id
\$ 0 F 3	+Id*Id\$	REDUCE T -> F
\$ 0 T 2	+Id*Id\$	REDUCE E -> T
\$ 0 E 1	+Id*Id\$	SHIFT + -> estado 6
\$ 0 E 1 + 6	Id*Id\$	SHIFT Id -> estado 5
\$ 0 E 1 + 6 Id 5	*Id\$	REDUCE F -> Id
\$ 0 E 1 + 6 F 3	*Id\$	REDUCE T -> F
\$ 0 E 1 + 6 T 9	*Id\$	SHIFT * -> estado 7
\$ 0 E 1 + 6 T 9 * 7	Id\$	SHIFT Id -> estado 5
\$ 0 E 1 + 6 T 9 * 7 Id 5	\$	REDUCE F -> Id
\$ 0 E 1 + 6 T 9 * 7 F 10	\$	REDUCE T -> T * F
\$ 0 E 1 + 6 T 9	\$	REDUCE E -> E + T
\$ 0 E 1	\$	ACCEPT